

Java Backend Architecture

Interview Series

Chapter 16.8

Container Registry

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

Chapter 16.8 – Container Registry

Container Registry Overview

A container registry stores, manages, and distributes OCI container images. Choosing the right registry affects security (image scanning, signing, access control), performance (pull speed, geographic distribution), and cost (storage, egress, pull limits).

Registry Comparison

Container Registry Comparison

Docker Hub Public/private repos 100 pulls/6h (anon) Auto-build Free tier available	AWS ECR IAM auth Lifecycle policies Trivy scanning Private VPC endpoints	GCP Artifact Reg Multi-format IAM + VPC-SC Auto scanning CMEK encryption	Harbor (OSS) Self-hosted RBAC + OIDC Replication Helm + OCI support
---	---	---	--

Registry	Auth	Scanning	Self-hosted	Best For
Docker Hub	Docker login / PAT	Docker Scout	No	Public OSS images
AWS ECR	IAM + aws ecr get-login	Amazon Inspector / Trivy	No	AWS-native apps
GCP Artifact Registry	gcloud auth / SA key	Container Analysis	No	GCP-native apps
Azure ACR	az acr login / service principal	Microsoft Defender	No	Azure-native apps
GitHub GHCR	GitHub PAT / OIDC	Dependabot	No	GitHub Actions CI/CD
Harbor	OIDC / LDAP / robots	Trivy / Clair	Yes	Enterprise on-prem
Quay.io (Red Hat)	Robot accounts	Clair	Yes	OpenShift / RHEL

Docker Hub Rate Limits & Workarounds

Anonymous pulls	100 pulls / 6 hours / IP. Shared IPs (NAT, CI/CD) hit this fast.
Authenticated free	200 pulls / 6 hours / Docker ID
Docker Pro/Team	5000 pulls / day
Workaround 1	docker login in CI/CD: authenticates with team account for higher limits
Workaround 2	Mirror base images to ECR/GCR/Harbor: docker pull -> tag -> push to internal registry
Workaround 3	Pull-through cache: registry mirrors Docker Hub, caches locally for org
Workaround 4	Use ghcr.io, quay.io for base images — generous or no rate limits

AWS ECR – IAM Auth Flow

AWS ECR – IAM Auth Flow + Lifecycle Policy



Token valid 12 hours. Use ECR credential helper for automatic renewal.

```
# Authenticate to ECR aws ecr get-login-password --region us-east-1 | \ docker login --username
AWS --password-stdin \ 123456789.dkr.ecr.us-east-1.amazonaws.com # Push image docker tag
myapp:1.0 123456789.dkr.ecr.us-east-1.amazonaws.com/myapp:1.0 docker push
123456789.dkr.ecr.us-east-1.amazonaws.com/myapp:1.0 # ECR Lifecycle Policy (keep last 10
tagged + delete untagged > 7 days) { "rules": [
{"rulePriority":1,"selection":{"tagStatus":"untagged",
"countType":"sinceImagePushed","countUnit":"days","countNumber":7},
"action":{"type":"expire"}}, {"rulePriority":2,"selection":{"tagStatus":"tagged",
"tagPrefixList":["v"],"countType":"imageCountMoreThan","countNumber":10},
"action":{"type":"expire"}} ] }
```

Harbor RBAC + Replication

Harbor – RBAC + Replication Architecture



Harbor projects isolate namespaces; quotas and scan policies per project

Image Tagging Strategies

Strategy	Example	Pros / Cons
latest tag	myapp:latest	Easy but mutable — never use in prod deployments
Semantic version	myapp:1.2.3	Human readable, immutable if not overwritten
Git SHA tag	myapp:abc1234	Traceable to commit, CI/CD friendly
SHA digest	myapp@sha256:abc...	Truly immutable — best for production pinning
Branch tag	myapp:main-20240601	Useful for PR preview environments
Environment tag	myapp:staging	Anti-pattern — mutable, no traceability

OCI Artifacts – SBOM, Signatures, Attestations

OCI registries store more than just container images. The OCI Distribution Spec supports arbitrary artifacts attached to image digests:

```
# Push SBOM as OCI artifact cosign attach sbom --sbom sbom.cyclonedx.json myimage:1.0 # Push
signature (Cosign) cosign sign --key cosign.key myimage:1.0 # Push SLSA provenance attestation
cosign attest --key cosign.key \ --predicate provenance.json \ --type slsaprovenance \
```

```
myimage:1.0 # Verify all artifacts cosign verify --key cosign.pub myimage:1.0 cosign  
verify-attestation --key cosign.pub --type slsaprovenance myimage:1.0
```

Pull-Through Cache (Registry Mirror)

```
# Configure Docker daemon to use a registry mirror # /etc/docker/daemon.json {  
"registry-mirrors": ["https://my-internal-mirror.company.com"], "insecure-registries": [] } #  
Harbor pull-through cache project: # Project type: Proxy Cache # Target:  
https://registry-1.docker.io # All docker pull <image> is transparently cached # ECR  
pull-through cache # Configure in ECR console: Mirror Docker Hub to ECR # Pull:  
123456789.dkr.ecr.us-east-1.amazonaws.com/docker-hub/library/alpine:3.19
```

Interview Q&A; – Container Registry

Q1. What is a container registry and how does it work?

A container registry is a service that stores, manages, and distributes OCI container images. It implements the OCI Distribution Specification (HTTP API). Clients push images (docker push) by uploading compressed layer blobs and a manifest. Clients pull images (docker pull) by fetching the manifest (by tag or digest), then downloading only missing layers. Registries are content-addressable — layers are identified by SHA256 digest and deduplicated. Public registries (Docker Hub, ghcr.io) allow open access; private registries require authentication.

Q2. What are Docker Hub rate limits and how do you work around them?

Docker Hub limits: anonymous (100 pulls/6h per IP), authenticated free (200 pulls/6h), Docker Pro (5000/day). In CI/CD environments, all agents may share a NAT IP, hitting anonymous limits quickly. Workarounds: (1) Authenticate with a service account: docker login in CI. (2) Mirror base images to ECR/GCR/Harbor — pull once, cache internally for unlimited org pulls. (3) Use ECR pull-through cache for automatic Docker Hub mirroring. (4) Use alternative registries for base images: ghcr.io, quay.io have no or generous limits.

Q3. How does AWS ECR authentication work?

ECR uses IAM for authentication. The flow: (1) Run `aws ecr get-login-password` which calls AWS STS to validate identity and returns a temporary token (valid 12 hours). (2) Pipe to `docker login --username AWS --password-stdin ecr_endpoint`. (3) Docker stores credentials locally and uses them for push/pull. In CI/CD: use ECR credential helper (`docker-credential-ecr-login`) for automatic token renewal. In ECS/EKS: the node role or task role grants ECR access without explicit login — IAM handles it transparently via the instance metadata service.

Q4. What are ECR lifecycle policies and how do you use them?

ECR lifecycle policies automatically expire old images to control storage costs. Rules specify: tagStatus (tagged/untagged), selection criteria (count or age), and action (expire). Example policy: (1) Expire untagged images older than 7 days. (2) Keep only the last 10 tagged images matching prefix 'v'. Rules are evaluated in priority order; first matching rule applies. Use cases: prevent unbounded storage growth, clean up CI/CD build artifacts, retain only production-relevant images. Run as dry run first with `--dry-run` to preview what would be deleted.

Q5. What is Harbor and what are its key features?

Harbor is an open-source, CNCF-graduated container registry for self-hosted environments. Key features: (1) RBAC with projects — isolate teams with granular roles (admin, developer, guest). (2) OIDC/LDAP/SAML authentication — integrates with enterprise identity providers. (3) Robot accounts for CI/CD — scoped access tokens without user accounts. (4) Vulnerability scanning (Trivy/Clair) with policy enforcement — block pull of vulnerable images. (5) Image replication — sync to/from ECR, GCR, Docker Hub. (6) Helm chart repository. (7) OCI artifact support. (8) Content Trust (Notary) and Cosign.

Q6. What is the difference between an image tag and an image digest?

A tag (`myimage:1.0`) is a mutable pointer — it can be updated to point to a different image. A digest (`myimage@sha256:abc...`) is an immutable, content-addressable reference — it uniquely identifies the exact image content. If you pull `myimage:1.0` twice and the tag was updated between pulls, you get different images. If you pull `myimage@sha256:abc...` you always get the exact same image. Production deployments should pin to digest for reproducibility. CI/CD: build, scan, tag with semver AND store the digest for deployment.

Q7. What is a pull-through cache and when should you use it?

A pull-through cache (registry mirror) acts as a transparent proxy in front of an upstream registry. When you pull an image, the cache checks if it has a local copy — if not, fetches from upstream, caches, and serves. Subsequent pulls are served from cache. Use cases: (1) Avoid Docker Hub rate limits for entire organization. (2) Reduce latency for international teams (local cache). (3) Provide air-gapped environments access to public images. (4) Audit and control which external images are used. Configure in Docker daemon: registry-mirrors or as a Harbor proxy cache project.

Q8. What OCI artifacts can be stored in a container registry besides images?

OCI registries support arbitrary artifact types via the OCI Artifact Specification: (1) Container images (traditional, manifest + layers). (2) Helm charts (stored as OCI artifacts since Helm 3.8). (3) Image signatures (Cosign stores them as .sig tags). (4) SBOMs in CycloneDX or SPDX format. (5) SLSA provenance attestations. (6) Vulnerability scan results. (7) Configuration policies (OPA Rego bundles). This makes the registry a content-addressable artifact store for the entire software supply chain.

Q9. How do you implement image scanning in a CI/CD pipeline?

Pipeline: (1) docker build to create image. (2) trivy image --exit-code 1 --severity CRITICAL,HIGH to scan — exit code 1 fails the pipeline. (3) If scan passes, docker push to registry. (4) Generate SBOM: trivy image --format cyclonedx --output sbom.json; attach to image. (5) Sign: cosign sign --key cosign.key myimage@sha256:... (6) Deployment validates signature before pull. Use GitHub Actions: aquasecurity/trivy-action@master or Docker Scout GitHub Action. Configure .trivyignore to acknowledge false positives with justification comments.

Q10. What is the GCP Artifact Registry and how does it differ from GCR?

Google Container Registry (gcr.io) is the legacy image registry. Artifact Registry is the modern replacement supporting multiple formats: Docker images, Helm charts, npm packages, Maven artifacts, Python packages. Artifact Registry improvements over GCR: regional endpoints (reduce latency), CMEK encryption, VPC Service Controls, fine-grained IAM (per-repo), automatic vulnerability scanning via Container Analysis, better audit logging. gcr.io now routes to Artifact Registry internally. New projects should use Artifact Registry with regional repos (e.g. us-central1-docker.pkg.dev).

Q11. What are robot accounts in Harbor and why are they used?

Robot accounts are service accounts in Harbor created for automation (CI/CD, deployment tools). They have: scoped permissions (push/pull to specific projects), no password expiration (token-based), revocable without affecting human users, and named for their purpose (robot\$ci-pipeline, robot\$deployer). Never use a human user account for CI/CD — if the employee leaves, all pipelines break. Robot accounts follow the principle of least privilege: CI pipeline needs push-only; deployment needs pull-only. Rotate robot tokens regularly and store in CI/CD secrets manager.

Q12. How do you implement a multi-region image distribution strategy?

Strategy: (1) Build once, push to primary region registry. (2) Replicate to regional registries (Harbor replication, ECR cross-region replication, GCR multi-region). (3) Deployments pull from the nearest regional registry. ECR cross-region: configure replication rules in ECR console to automatically sync images to destination regions. Harbor: create replication rules with trigger (on-push or scheduled). Benefits: lower pull latency, resilience against regional outages, egress cost reduction (pull from same-region registry is free in most clouds).

Q13. What is SLSA provenance and how does it relate to container registries?

SLSA (Supply-chain Levels for Software Artifacts) is a framework for supply chain security. Provenance is metadata about how an artifact was built: what source commit, what build system, what dependencies. Generated during CI/CD and stored as an OCI attestation: `cosign attest --predicate provenance.json --type slsaprovenance myimage`. SLSA levels: Level 1 (provenance exists), Level 2 (hosted build system, signed provenance), Level 3 (hardened build system), Level 4 (hermetic builds). Verifiers can check provenance before deployment: does this image come from the expected repo and CI system?

Q14. How do you set up a private ECR repository with fine-grained access?

ECR repositories have two types of policies: (1) Repository policy (resource-based): attached to the repo, controls cross-account access. Example: allow account 987654321 to pull. (2) IAM policy (identity-based): attached to user/role, controls what repos they can access. For CI/CD: create an IAM role with `ecr:GetAuthorizationToken`, `ecr:BatchGetImage`, `ecr:GetDownloadUrlForLayer`, `ecr:BatchCheckLayerAvailability`, `ecr:PutImage`. For ECS/EKS: task execution role needs `ecr:GetDownloadUrlForLayer` and `ecr:BatchGetImage` — AWS handles auth automatically via instance metadata.

Q15. What is image promotion and how do registries support it?

Image promotion is the practice of moving an image through environments (dev → staging → prod) by re-tagging or copying (not rebuilding). The same artifact tested in staging is deployed to prod. Pattern: CI builds `myapp:git-sha123`, testing validates it, promotion adds tags: `myapp:staging-20240601`, `myapp:v1.2.3`, `myapp:prod`. Use `docker tag` + `docker push` for same-registry promotion. For cross-registry: `crane copy` or `skopeo copy` (copies without pulling locally). Harbor replication can automate promotion-based distribution to regional registries on tag event.

Q16. How do you handle secrets and credentials for registry access in Kubernetes?

Kubernetes uses `imagePullSecrets` for private registry access. Create secret: `kubectl create secret docker-registry regcred --docker-server=myregistry.io --docker-username=user --docker-password=token`. Reference in Pod: `imagePullSecrets: [name: regcred]`. Better alternatives: (1) AWS ECR: use ECR credential helper or IRSA (IAM Roles for Service Accounts) — EKS nodes pull ECR images without explicit secrets via node IAM role. (2) GKE Workload Identity: pods assume GCP service account with Artifact Registry reader role. (3) Vault dynamic credentials: inject fresh pull tokens via Vault Agent.